

SQLite-Carver-Pro v1.9.2

Technical Reference Manual & Low-Level Forensic Engineering Specifications

 SEE & SQLCipher Decryption, WAL Slack Carving, Multi-Version & Anti-Forensics

Comprehensive technical manual detailing physical page layout mechanics, on-the-fly decryption of SQLCipher and SQLite Encryption Extension (SEE) databases, WAL frame payload decryption, superseded frame recovery (WAL Slack Space), multi-version page history, and PRAGMA secure_delete heuristics.

Author: Dam-FOR3K

Version: 1.9.2 (Professional Forensic Edition)

Target Architecture: Standalone Air-Gapped Engine (Windows x86_64, Linux, macOS)

Methodological Standard: ISO/IEC 27037 Digital Evidence Handling & Forensic Soundness

1. Physical SQLite Architecture & Forensic Audit Areas

Unlike standard database management suites that depend on the official SQLite engine library (`sqlite3.dll`), **SQLite-Carver-Pro** operates at the binary level directly on raw database disk pages. This low-level approach bypasses database engine abstraction layers, allowing the direct recovery of unallocated, overwritten, fragmented, and anti-forensically concealed data.

1.1 The 100-Byte Database Header

The first 100 bytes of a database file (located exclusively on Page 1) define the global database parameters. Each field holds critical forensic value:

Offset	Size	Field Name	Forensic Significance
0	16 B	Magic Header String	Mandatory ASCII string: <code>SQLite format 3\0</code> . Validates authentic file signature.
16	2 B	Page Size	Physical page size (512 to 65,536 bytes, power of 2). Dictates all boundary and offset math.
18	1 B / 1 B	Write / Read Version	File format version (1 = Conventional Rollback Journal, 2 = Write-Ahead Log / WAL mode).
20	1 B	Page Reserved Space	Critical Area: Number of reserved bytes at end of each page (0 to 255). Ignored by standard B-Tree logic. Stores IV/HMAC in SQLCipher (48 bytes) or serves as an anti-forensic steganography vector.
24	4 B	File Change Counter	Incremented on each write transaction. Detects tampering and rollback anomalies.
28	4 B	Database Size in Pages	Number of valid pages. Immediately exposes <i>EOF Slack</i> (data appended beyond database bounds).
32	4 B	First Freelist Trunk Page	Pointer to the primary trunk page of the unallocated freelist chain.
36	4 B	Total Freelist Pages	Total count of decommissioned pages available for immediate reuse.
52	4 B	Largest Root Page / Vacuum	Auto-Vacuum mode identified by SQLite-Carver-Pro v1.9:2NONE , <code>FULL</code> , or <code>INCREMENTAL</code> .
56	4 B	Text Encoding	Default string serialization (1 = UTF-8, 2 = UTF-16le, 3 = UTF-16be).

1.2 B-Tree Page Types

Every physical database page is governed by one of four B-Tree page types identified by its first byte:

- **Table Leaf Page (`0x0D`):** Stores actual application records (payload cells) and the integer RowID. Primary target for cell carving.
- **Table Interior Page (`0x05`):** Routing B-tree containing navigation keys and child page pointers.
- **Index Leaf Page (`0x0A`):** Contains indexed column keys and row references. Also houses `WITHOUT ROWID` tables.
- **Index Interior Page (`0x02`):** Index routing B-tree pointing to child index pages.

2. Unified Cryptographic Engine: SEE & SQLCipher

2.1 Cryptographic Architecture Comparison (SEE vs SQLCipher)

In embedded, enterprise, and mobile forensics, investigators frequently encounter encrypted SQLite databases:

Feature	SQLCipher (v3 / v4)	SQLite Encryption Extension (SEE)
Cipher & Mode	AES-256-CBC (per-page initialization vector)	AES-256-OFB or AES-128-OFB (Output Feedback)
Key Derivation	PBKDF2-HMAC-SHA512 (256,000 iter) or SHA1 (64,000 iter)	Direct SHA-256 / SHA-1 derivation or raw hex key
Reserve Bytes	48 bytes (16B IV + 20/32B HMAC)	0 or 16 bytes (nonce stored at end of page)
Page 1 Header	Bytes 0-15 overwritten by cryptographic salt	Bytes 16-23 stored in plaintext (geometry preserved)
WAL Encryption	Frame page payloads encrypted with AES-CBC	Frame page payloads encrypted with AES-OFB

2.2 Forensic Fingerprinting of SEE Databases

Because the SQLite Encryption Extension (SEE) preserves bytes 16–23 in plaintext so the pager can determine page size, **SQLite-Carver-Pro v1.9.2** automatically detects SEE databases with **98% confidence**:

- Bytes 0–15 exhibit high entropy (encrypted magic string).
- Bytes 16–23 exhibit standard SQLite geometry: valid power-of-two page size (512–65536), write/read versions 1 or 2, and payload fractions `64, 32, 32`.
- This structural diagnostic alerts the investigator that the file is an authentic SEE database rather than random unallocated noise.

2.3 Transparent Decryption of Encrypted WAL Frames

In both SQLCipher and SEE, SQLite writes the 32-byte WAL header and 24-byte frame headers in plaintext. However, the page payload (e.g. 4096 bytes) is encrypted.

In **v1.9.2** `WalDiffEngine` derives the master key and automatically decrypts each frame payload in memory using `decrypt_wal_frame_page()`. This allows complete carving of active WAL records, WAL Slack space, and transaction mutations without modifying the evidentiary file.

3. Advanced Low-Level Forensic Innovations

3.1 WAL Slack Space Carving (Superseded Frames)

When SQLite executes a checkpoint, the WAL file is often logically reset by changing the transaction salt (`salt1`) rather than physically zeroing or truncating the file. Frames situated beyond the current active transaction sequence that retain older salts constitute the **WAL Slack Space**.

These superseded frames contain pre-checkpoint historical records. **SQLite-Carver-Pro v1.9.2** parses past active checkpoint boundaries and extracts all records with provenance labeled as `wal_slack`.

3.2 Multi-Version Page Reconstitution

Within a single WAL journal or across checkpoint boundaries, the same physical page may be written multiple times. The engine tracks all frame versions of each page, providing a chronological timeline of row modifications.

3.3 Anti-Forensics: PRAGMA secure_delete Detection

When a database operates with `PRAGMA secure_delete = ON`, deleted cells are zeroed out (`0x00`) before being linked into freeblocks. The **v1.9.2** integrity engine analyzes the zero-byte ratio across freeblocks: if more than **90% of bytes are zeroed**, it flags `PRAGMA_SECURE_DELETE_DETECTED`, alerting the investigator to deliberate anti-forensic data wiping.

4. CLI Command Reference & Desktop GUI

4.1 Modern Desktop GUI Application

```
# Launch modern interactive forensic desktop GUI
sqlite-carver gui

# Open encrypted database directly
sqlite-carver gui evidence.db --key "PassphraseOrHexKey"
```

4.2 CLI Command Reference

```
# 1. Complete forensic carve with transparent decryption and HTML report
sqlite-carver carve evidence.db --key "secret_passphrase" --export report.html

# 2. Extract records from WAL Slack space (superseded frames)
sqlite-carver wal-diff evidence.db evidence.db-wal --slack --key "secret_passphrase"

# 3. Raw Page Hunter on unallocated memory dump without SQLite header
sqlite-carver carve-raw memory.dump --workers 8 --export carved.sqlite

# 4. Deep search across records, embedded BLOBs, and WAL
sqlite-carver search evidence.db "target_keyword" --key "secret_passphrase"
```

Certified Standalone Air-Gapped Distribution

SQLite-Carver-Pro is packaged as a single self-contained 64-bit binary (`sqlite-carver-v1.9.2.exe`) designed for deployment on isolated forensic workstations without Python dependencies.